

CODE REFERENCE

Simulator Core — Line-by-Line Walkthrough

starter/envs/ · the corridor loop, observation vector, reward library and MARL
glue

Group B3 · UST Electronics Engineering

An Evaluation of Multi-Agent Reinforcement Learning for
Dynamic Bus Scheduling Under Non-Ideal Conditions

Adviser: Asst. Prof. Kanny Krizzy D. Serrano, MSc

Code walkthrough — the simulator (starter/envs/ , starter/agents/)

Line-by-line explanation of the simulation core: the corridor loop every experiment drives, the MARL glue, the observation vector and the reward library.

Generated with docs/prompts/Code_Walkthrough_Prompt.md . Line numbers current as of 2026-09-12.

File index

File	Lines	What breaks if you delete it
envs/corridor_sim.py	195	Everything. Every baseline and every MARL run calls <code>simulate()</code>
envs/reward.py	68	The agent has no score. Also holds <code>Q_REF</code> , which <code>obs.py</code> imports
envs/obs.py	39	The agent has no input
envs/marl_env.py	63	The network can't be used as a controller
agents/ddqn.py	131	No learning
baselines/even_headway.py	13	Nothing — <code>corridor_sim</code> has its own EH at L54
envs/bus_env.py	77	Nothing. Confirmed zero callers.

⚠ Read this first

Three things in this code do not match what the manuscript says. They are detailed in the findings, but in short:

1. **The skip action does nothing.** `corridor_sim` discards it (L134).
2. `H0 = 300` (L34) — the real 2021 scheduled headway is 600 s.
3. `bus_env.py` **is dead** — 77 lines, no callers.

envs/corridor_sim.py — the heart

One loop that any controller drives. A controller is just a function `decide(obs) -> (hold_seconds, skip)` . NC, FH, EH and the MARL policy are all different `decide` functions running on an identical environment — which is what makes the comparison fair by construction.

L17–21 — imports and SUMO bootstrap

```
L18-19 if "SUMO_HOME" in os.environ: sys.path.insert(0, ../tools)
      WHY   traci and sumolib ship inside the SUMO install, not on pip
      WATCH without SUMO_HOME set, L20 fails at import time
```

L23–33 — load the corridor from disk

```
L23  STOPS = [...corridor.txt...]           the 26 modelled stops, in order
L24  EDGES = ["e0", "e1", ...]             SUMO edge names, one per stop
L25-26 read stop_coordinates.csv and stops.csv, reindexed to STOPS order
      WATCH .loc[[int(s) for s in STOPS]] will KeyError if corridor.txt names a
          stop that is not in the CSVs. This is how a bad corridor edit fails.

L27-30 equirectangular projection, then straight-line distance between stops
      WHAT  DIST[i] = metres from stop i to stop i+1
      WHY   the RESULTS run on a schematic corridor carrying these real
          distances. The real-geometry net exists but is used by watch.py for
          visual verification only.

L31  SEGV[i] = DIST[i] / run_s[i]           the calibrated segment speed
L32  BASE = per-stop dwell, floored at 8 s
L33  DEM   = per-stop mean boardings
```

L34–35 — the parameter block

```
L34  H0, NBUS, CVD, CAP, BIG, TBREAK, ETA = 300.0, 12, 0.25, 0.4, 600.0, 400.0, 0.8
      H0      scheduled headway (s)          <-- SHOULD BE 600. See the change list.
      NBUS    buses launched                  12
      CVD     dwell noise CV                  0.25 (lognormal)
      CAP     max hold as a fraction of H0    0.4
      BIG     placeholder stop duration       600 s, overwritten per stop
      TBREAK  breakdown immobilisation       400 s
      ETA     weather intensity CV            0.8
      WATCH  everything scaled by H0 moves when you fix it: the FH/EH caps, the
          MARL dT, and TBREAK's severity RELATIVE to headway (1.33xH0 now,
          0.67xH0 at the correct value).

L35  FIXED_DWELL, BOARD_S, MAX_SERV, CAP_PAX = 6.0, 4.0, 90.0, 60
      BOARD_S  4 s per boarding passenger – a constant, not fitted
      CAP_PAX  60 – asserted, not sourced (observed max_load is 77)
```

L36–45 — schedule and control stops

```
L37-39 CUM[i] = nominal arrival time at stop i (running time + dwell, cumulative)
      used only to time the passenger injection windows

L45  CONTROL_STOPS = [0, 1, 5, 17, 20]     bs_ids 5280, 5857, 5859, 5867, 4046
      WHY   derived from the four §3.2.2 criteria, NOT evenly spaced
      WATCH these are POSITIONAL indices into corridor.txt. Add or remove a stop
          and they silently point somewhere else. This is the trap if 6361
          (Slaughter Station) is restored.
```


L104–152 — the main loop

```
L98  departs = {"b0": 0, "b1": H0, "b2": 2*H0, ...}
      WHAT  buses launch one headway apart
      WATCH at H0=300 x 12 buses = 3600 s of departures against a ~5000 s run,
            so all 12 overlap. At H0=600 only ~8 would. This is the
            H0/NBUS/run-time consistency problem.

L104  while t < H0*NBUS + 30000 and (vehicles remain or buses still to launch)
L107  traci.vehicle.add(...)          launch when t reaches the departure
L112-114 setBusStop(v, stop, duration=BIG) for all stops
      WHY   register every stop up front with a long placeholder duration; the
            real duration is set when the bus actually arrives (L138)

L119  if st and not prev[v] and i < ns:      <-- ARRIVAL EDGE DETECTION
      WHAT  fires once, on the transition from moving to stopped
      WHY   isStopped() is true for many consecutive steps; without the
            `not prev[v]` guard every metric would be counted repeatedly

L120  arr[s].append(t); barr[(bi, i)] = t    record the arrival
L122  nwait = traci.busstop.getPersonCount(s)
L124  dserv = min(MAX_SERV, FIXED_DWELL + BOARD_S*nwait) * DWN[bi, i]
      WHAT  demand-responsive dwell: 6 s + 4 s per waiting passenger, capped at
            90 s, times this bus-stop's noise draw
      WHY   this is the feedback that CREATES bunching – a late bus finds more
            people waiting, dwells longer, falls further behind
```

L126–135 — the control decision

```
L126  if i in control_stops and bi > 0:
      WATCH bi > 0 means the LEAD BUS NEVER ACTS. It has no bus in front, so
            no forward headway. 11 of 12 buses are controllable.

L127  hf = t - barr[(bi-1, i)] if known else H0
      WHAT  forward headway = time since the bus ahead reached THIS stop

L128  hb = barr[(bi, i-1)] - barr[(bi+1, i-1)] if known else H0
      WHAT  backward headway ESTIMATED from the PREVIOUS stop
      WHY   the follower has not reached this stop yet, so its gap here is not
            observable – it is inferred from one stop back. The manuscript calls
            this the "estimated backward headway" for this reason.

L131  w = WF[bi, i] if W else 1.0            weather intensity for the obs vector
L132-133 obs = dict(hf, hb, load, queue, idx, n, H0, cap, bus, w, b)
      WHAT  11 keys; obs.py turns them into the 7-vector
L134  hold, _skip = decide(obs)
      WATCH *** THE SKIP RETURN VALUE IS DISCARDED. *** See findings.
L135  hold = clip(hold, 0, 0.4*H0)           the cap applies to every controller
```

L136–152 — apply and resume

```
L136  tb = TBREAK if (B and bi == bk_idx and i == bks) else 0
L137  if tb > 0: bk_triggered = True          so later obs carry the breakdown flag
L138  target[v] = max(FIXED_DWELL, dserv) + hold + tb
      WHAT  total time this bus sits: service + control hold + breakdown

L141-143 when (t - arrival) >= target, resume
L144-150 after resuming, set the segment speed for the upcoming leg
      L146  if W: f *= WF[bi, i]              weather slows
      L147  if T: f /= TF[bi, i]             traffic scales both ways
      L149  setMaxSpeed(max(2.0, SEGV[i] / f))
      WATCH floor of 2 m/s stops a severe weather draw freezing a bus entirely

L151  if (not st) and prev[v]: idx[v] += 1   departure edge -> advance the index
```

L157–185 — the metrics

```
L157  cvs = [std(diff(sorted(arr[s]))) / mean(diff(...)) for s in STOPS[1:] if >=3 arrivals]
      WHAT  headway CV per stop, then averaged -> the headline metric
      WATCH STOPS[1:] excludes the origin, where headways are the launch
            schedule by construction

L158  tt = [completion - entry]              travel time per bus

L161-165 wait_model = boardings-weighted ( $E[H]/2)(1+CV^2)$ 
      WHY   expected wait under random passenger arrivals given the REALIZED bus
            headways. This is the PRIMARY wait metric.
      WATCH it is a model, not a measurement. The manuscript's phrasing implies
            simulated per-passenger wait; this is an analytic formula.

L168-173 wait_direct = SUMO's own per-passenger waitingTime from tripinfo
      WHY   independent cross-check. Matches wait_model under mild conditions;
            inflates under heavy weather because far-stop passengers are injected
            before any bus can reach them.

L174-176 delete the temp files
L181-184 if trace: per-bus (arrival_time, stop_index) + cumulative distance
      WHY   this is what marey.py draws the time-space diagram from
```

Findings — corridor_sim.py

The skip action is not implemented	L134 discards it (<code>_skip</code>). The docstring at L12–13 says it "takes effect if the caller enabled skipping (<code>SKIP_ENABLED</code>)" — <code>SKIP_ENABLED</code> does not exist anywhere in the repository. Confirmed by grep. So actions 5–9 of the 10-action space are behaviourally identical to 0–4 inside the simulator. <code>marl_env</code> currently masks skip to 0 anyway (<code>skip_enabled=False</code> by default), so nothing is silently wrong <i>today</i> — but the moment anyone sets <code>skip_enabled=True</code> , the agent will emit skips that the simulator ignores, and the resulting "skip does nothing" will look like a reward-tuning problem. Fix before MSA2 enables skipping.
<code>H0 = 300</code>	L34. Should be 600. See <code>docs/planning/GTFS_FINDINGS_CHANGE_LIST_2026-09-12.md</code>
<code>CONTROL_STOPS</code> are positional	L45. Restoring stop 6361 shifts every index after it
The lead bus never acts	L126 (<code>bi > 0</code>). Correct — it has no leader — but it means 11 of 12 buses are controlled, which is worth stating in the manuscript
<code>__main__</code> parity targets may be stale	L190 expects "NC~0.335, FH~0.153, EH~0.172" for the all-interior configuration. The committed <code>mc_summary.md</code> reports 0.331 / 0.237 / 0.271 at the five designated stops. Different configurations, so not necessarily wrong — but UNVERIFIED , and worth re-running before quoting either

envs/obs.py — what a bus sees

39 lines. Turns the 11-key obs dict into the manuscript's 7-vector (Table 3.6), normalised so every feature sits near [0, 1].

L24	<code>idx/(n-1)</code>	where along the corridor
L25	<code>hf/H0</code>	forward headway; 1.0 means exactly on schedule
L26	<code>hb/H0</code>	estimated backward headway
L27	<code>load/cap</code>	occupancy
L28	<code>queue/Q_REF</code>	waiting passengers, <code>Q_REF = 20</code> from <code>reward.py</code>
L29	<code>(w - 0.5)/2.5</code>	weather; the WF clip is [0.5, 3.0], so this maps to [0, 1]
L30	<code>b</code>	breakdown flag, already 0/1

WATCH L25-26 divide by `H0`. Changing `H0` to 600 HALVES these two inputs, so the network sees a different input distribution and the `gate1` weights cannot be fine-tuned – retrain from scratch. (No checkpoint was saved anyway.)

WATCH L16 imports `Q_REF` from `reward.py`. The observation module depends on the reward module for a normalisation constant. Harmless, but it means the two cannot be swapped independently.

Findings — obs.py

None beyond the `H0` coupling above.

envs/reward.py — how the agent is scored

The manuscript fixes the *structure* — three non-positive penalties — and leaves the expressions and weights as the implementation deliverable. So this file is a **menu of candidates**, and `Config` picks which.

```
L17-21 decode_action(a, H0, dt)
    alpha = (a % 5) * 0.1      -> {0, .1, .2, .3, .4}
    skip  = a // 5            -> 0 or 1
    returns (alpha * dt, skip)
    WATCH at dt=300 the holds are {0,30,60,90,120} s; at dt=600 they become
           {0,60,120,180,240}. With TBREAK=400 s, the current maximum hold
           cannot begin to correct a breakdown. This is the leading suspect
           for the gate1 plateau.

L25-30 THREE candidate irregularity terms
    irr_dev    squared deviation of hf from H0          (schedule adherence)
    irr_even   squared hf/hb asymmetry                 (even spacing)
    irr_both   average of both

L32-35 TWO candidate wait terms
    wait_queue waiting riders x the headway they endured (at-stop)
    wait_hold   holding delay x onboard load             (in-vehicle)
    WATCH these penalise OPPOSITE things. wait_queue rewards holding to even
           out gaps; wait_hold punishes holding. Which you pick materially
           changes what the agent learns.

L37-40 TWO candidate skip terms
    skip_stranded skip x queue/Q_REF      (demand-aware)
    skip_flat     flat penalty per skip
    WATCH both are currently unreachable – see the corridor_sim finding

L47-57 compose(prev, cur, a_prev, cfg) = -(w1*irr + w2*wait + w3*skip)
    WHY the reward for an action is computed at the bus's NEXT decision,
        using the state that action produced. That is the semi-MDP
        assembly – the action's consequence is not observable until the
        bus reaches the next control stop.
```

Findings — reward.py

- `Q_REF = 20.0` (L14) is labelled "tunable scale constant" and has never been tuned. It scales both the queue observation and the wait penalty.
- The candidate menu is the EO2.1 deliverable and nothing is committed — that is by design, not an omission.

envs/marl_env.py — the glue

```
L20-36 @dataclass Config – every experiment knob in one place
L26   H0: 300.0; dt: 300.0          <-- both need to become 600
L33   control_stops = (0,1,5,17,20)
L29   eps_decay = 30_000           in TRANSITIONS, not episodes

L46-59 Mar1Controller.__call__(obs) – this IS the decide() function
L49-55 if this bus has acted before, compute the reward for that action
      now (state `obs` is what it produced), push the transition, learn
L56   choose a new action (greedy when not training)
L57   remember (obs_vec, action, obs_dict) for this bus
L59   return hold, and skip only if skip_enabled
WHY   parameter sharing: one agent instance serves every bus, keyed by
      obs["bus"]

L61-63 finalize() – clear the per-bus memory at episode end
WHY   the last action of an episode has no next state to bootstrap from,
      so it must be dropped rather than paired with the next episode's
      first observation
WATCH forget to call this between episodes and you corrupt the buffer
      with cross-episode transitions
```

Findings — marl_env.py

- `Config.skip_enabled` defaults to `False`, which is currently the *only* thing preventing the discarded-skip bug from mattering.
-

envs/bus_env.py — dead

77 lines. A PettingZoo AEC environment skeleton, superseded by `corridor_sim.simulate()`. **Grep confirms zero importers.** Delete it, or move it to an `archive/` folder with a note — leaving it invites someone to build on the wrong base.

baselines/even_headway.py — redundant

13 lines implementing Even-Headway. `corridor_sim.py:54` has its own `eh_hold`, and that is the one the experiments use. This file is imported by nothing. Two implementations of the same rule is exactly the drift risk we removed from the cleaning rules — consolidate or delete.

Consolidated findings

Severity	Finding	Where
High	Skip action discarded; <code>SKIP_ENABLED</code> never existed	<code>corridor_sim.py:134</code> , docstring L13
High	<code>H0 = 300</code> , should be 600	<code>corridor_sim.py:34</code> , <code>marl_env.py:26</code>
Medium	<code>wait_queue</code> and <code>wait_hold</code> penalise opposite behaviours	<code>reward.py:32-35</code>
Medium	<code>CONTROL_STOPS</code> are positional indices	<code>corridor_sim.py:45</code>
Low	<code>bus_env.py</code> — 77 lines, zero callers	<code>envs/bus_env.py</code>
Low	<code>baselines/even_headway.py</code> duplicates <code>corridor_sim.py:54</code>	both
Low	<code>Q_REF</code> never tuned; couples obs to reward	<code>reward.py:14</code> , <code>obs.py:16</code>
UNVERIFIED	<code>__main__</code> parity targets vs the committed MC table	<code>corridor_sim.py:190</code>